

3D Tank Box — C++／DirectX 11による3D戦車ゲーム開発

1. プロジェクト基本情報

項目	内容
プロジェクト種別	入社時研修の一環として行ったチーム制作
開発期間	2020年5月～7月（約1.5か月）
チーム構成	プログラマー4名
ゲームジャンル	三人称視点の3D戦車アクションゲーム
開発言語	C++、HLSL
主な技術	Win32、DirectX 11、DirectInput、FreeType
開発環境	Visual Studio、GitLab
主な個人担当	エンジン／ゲームフレームワーク、レンダリング、全Shaderの設計・実装、FreeTypeによる文字描画、3C、Gameplay、粒子システム、簡易カットシーン制御、レベル設計・実装

ソースコード：GitHub - 3DTank (<https://github.com/aleidi/3DTank>)

実機動作デモ動画：実機動作デモ動画.mp4

2. プロジェクト概要

本プロジェクトでは、入社直後の技術研修として、4名のチームで3D戦車アクションゲームを制作しました。開発期間は2か月未満で、既製ゲームエンジンを使用せず、C++とDirectX 11を用いてゲームの実行基盤から構築することが課題でした。

研修課題には、プレイヤーの移動・照準・射撃、敵AI、障害物との衝突、パーティクル、サウンド、メニュー、多言語対応、カットシーン、簡易レベル編集などが含まれていました。限られた期間内に、要件の達成度とコード構造の両方が評価対象となるプロジェクトでした。

チームとして、タイトル画面から戦闘、ボス戦、ゲーム終了まで進行できるゲームを完成させました。私は、低レイヤーの描画基盤から実際のプレイ体験までをつなぐ部分を主に担当しました。

3. 本人の担当範囲

3.1 主に設計・実装した範囲

- Win32アプリケーションとゲームのメインループ

- **GameObject**、**Component**、**Transform**、**SceneManager**を中心としたオブジェクト／コンポーネント基盤
- DirectX 11の初期化、描画リソース管理、描画処理の抽象化
- Mesh、Shader、Texture、Constant Buffer、Pipeline Stateを組み合わせた描画フレームワーク
- 基本描画、ライティング、法線マップ、Skybox、UI、Billboard、Particle、VFXを含む、プロジェクト内の全HLSL Shaderの設計・実装
- FreeTypeによるフォント読み込み、字形のテクスチャ生成および文字描画
- プレイヤー入力、戦車操作、砲塔制御、カメラ、照準、射撃を含む3CおよびGameplay
- Geometry Shaderを使用した簡易粒子システム
- CSVのキーフレームからCamera Transformを再生する簡易カットシーン制御
- UI、サウンド、VFXをGameplayへ接続する処理
- 1ステージ分のレベル設計、敵配置、イベントおよび演出の組み込み

3.2 他メンバーの実装と連携した範囲

- AI担当者が実装したAI基盤を、敵配置、起動タイミング、戦闘進行、ボス戦演出と連携しました
- 他メンバーが利用するためのゲームオブジェクト、描画、レベル管理などの共通基盤を提供しました
- チームの実装を一つのゲームとして動作させるため、各機能の結合と不具合修正を行いました

3.3 主担当ではない範囲

- AIの基礎モジュール
- 衝突・物理判定の基礎実装
- Dear ImGuiを使用した編集機能
- 3Dモデル、テクスチャ、画像、音源などの素材制作

3Dモデル、テクスチャ、画像、BGM、効果音などの素材は、インターネット上で公開されていた素材を収集して使用しました。これらの素材制作は個人の成果には含めていません。一方、素材をゲームへ読み込み、描画、再生し、Gameplayや演出へ組み込むための実装は担当しました。その他の上記項目はチームメンバーまたは第三者が担当した領域です。

4. システム全体の構成

```
Win32 Application
↓
Engine Main Loop
├─ Input Update
├─ Fixed Update / Update / Late Update
├─ GameSystem / Level / GameMode
├─ Rendering
└─ Deferred Object Destruction
    ↓
SceneManager
    └─ GameObject
        ├── Transform
        ├── RenderComponent
        ├── ScriptComponent
        ├── MovementComponent
        └─ Sound / Collision Component
            ↓
RenderManager
    └─ Drawable + Bindable
        ├── Vertex / Index Buffer
        ├── Shader / Texture / Sampler
        ├── Constant Buffer
        └─ Blend / Depth-Stencil / Rasterizer State
            ↓
DirectX 11
```

UnityやUnreal Engineを使用した経験から、オブジェクトとコンポーネントの組み合わせ、ライフサイクル、Controller・Pawn・Cameraの責務分割などを参考にしました。ただし、その構造をそのまま再現するのではなく、研修の規模と期間に合わせ、必要な機能をC++上で自ら設計・実装しました。

5. 代表的な技術課題と実施内容

5.1 DirectX 11によるレンダリング基盤の構築

目的

既製ゲームエンジンを使用せず、3Dモデル、UI、パーティクルなどを描画できる基盤を構築する必要がありました。また、Gameplay側がDirectX 11のAPIを直接操作せずに描画機能を利用できる構造が必要でした。

課題

本プロジェクトで初めて低レイヤーのグラフィックスAPIを扱ったため、Device、DeviceContext、SwapChain、Render Target、Depth-Stencil、各Shader Stageなど、描画パイプライン全体を理解しながら実装する必要があり

ました。さらに、描画リソースの種類が増えても、各Meshに同じDirectX 11の処理を重複させない設計が求められました。

判断と設計

- GPUリソースとPipeline Stateを共通の**Bindable**として扱い、**Drawable**が描画前に必要な要素を順にBindする構造としました
- ゲームオブジェクトと描画データの間に**RenderComponent**と**Mesh**を配置し、Gameplayと描画APIの依存を分離しました
- 不透明オブジェクト、Skybox、透明オブジェクト、VFX、Particle、3D UI、2D UIの描画順を管理しました
- FreeTypeでフォントから字形を読み込み、DirectX 11のテクスチャとして生成・描画する文字表示機能を実装しました

実装内容

- DirectX 11の初期化と、SwapChain・描画先・深度バッファの構築
- Buffer、Shader、Textureなど、DirectX 11の描画リソースを扱うラッパーの実装
- View／Projection行列とカメラ情報の描画処理への受け渡し
- プロジェクト内の全HLSL Shader（Vertex／Pixel／Geometry）の実装

結果

モデル描画からUI・VFXまで、ゲームに必要な描画機能を同一の基盤上で動作させました。この経験により、エンジンの描画機能を利用するだけでなく、Draw Callに至るまでのリソース生成、状態設定、Shader実行の流れを具体的に理解しました。

5.2 ゲーム／エンジンフレームワークの設計

目的

4名が並行して機能を実装するため、個別のオブジェクトが入力、描画、サウンド、衝突などを直接管理する構造を避け、共通の更新単位と機能追加方法を用意する必要がありました。

判断と設計

- **GameObject**を機能の入れ物とし、描画や移動などを**Component**として追加する構成を採用しました
- **Start**、**FixedUpdate**、**Update**、**LateUpdate**に相当する更新タイミングを定義しました
- 車体、砲塔、カメラなどの複合構造を表現するため、**Transform**に親子階層を持たせました
- ゲーム固有の進行を**GameSystem**、**GameInstance**、**Level**、**GameMode**へ分けました

結果

描画基盤、共通オブジェクト、ゲームルール、個別Gameplayを段階的に分離し、チームメンバーが共通基盤上で機能を追加できる状態を作りました。また、UnityやUnreal Engineで利用していた概念について、内部で必要となる責務とデータの流れを考える機会になりました。

5.3 3Cと戦車Gameplayの実装

目的

プレイヤーが戦車を移動させ、周囲を見回し、狙った対象へ射撃できる一連の操作を実現する必要がありました。

設計と実装

- 入力取得、**PlayerController**、**PlayerTank**、**PlayerCamera**を分け、入力と制御対象の責務を整理しました
- 車体、砲塔、カメラ追従点をTransformの親子関係で構成しました
- マウス入力による視点回転、俯仰角制限、ADS／非ADS時のFOV補間、カメラ距離の調整を実装しました
- プレイヤーからカメラ候補位置へRaycastを行い、環境との交差位置にカメラを補正することで壁へのめり込みを抑制しました
- カメラ正面のRaycastで照準対象を取得し、実際の砲口位置から弾を発射する処理を実装しました
- 発射間隔、砲塔の追従、弾の生成、発射／着弾Particle、Sound、HUDフィードバックを接続しました

結果

入力、Camera、Tank、Projectile、VFX、Sound、UIを一連のプレイ体験として統合しました。単独の機能実装だけでなく、異なるシステム間で座標、状態、実行順を整合させる経験を得ました。

5.4 Geometry Shaderを利用した簡易粒子システム

目的

砲撃、着弾、爆発などの戦闘演出を共通の仕組みで再生できるようにする必要がありました。

判断と設計

ParticleSystemをエフェクトの再生単位とし、各インスタンスが1つのEmitter設定、粒子データ、再生状態、描画リソースを保持する構成としました。Emitterでは点またはBox範囲からの発生位置、発生数、初速度などを設定し、System側で粒子の生成・再利用、寿命管理、更新、描画までを管理しました。

CPU側で粒子属性を更新してPoint ListとしてGPUへ渡し、Geometry Shaderで各Pointをカメラ方向へ向く四角形へ展開しました。これにより、粒子の発生・シミュレーションとBillboard描画の責務を分けました。

処理構成

SceneManagerがParticleSystemを生成



Emitter設定から粒子を生成・再利用



CPUで寿命・位置・速度・Spriteを更新



動的Vertex BufferへPoint Listとして転送



Geometry ShaderでBillboardへ展開して描画

実装内容

- **ParticleSystem**によるEmitter設定、粒子データ、再生状態、描画リソースの一元管理
- 点／Box範囲のEmitter、粒子の寿命・発生数・速度・再生時間・Loopの設定
- 使用済み粒子の再利用と、Sprite SheetのUVアニメーション
- 動的Vertex BufferとGeometry Shaderを用いたBillboard描画
- 発射、着弾、敵撃破、環境演出への共通Systemの組み込み

結果

同じ粒子基盤を複数の戦闘・環境演出へ利用できるようにしました。CPU側のシミュレーションとGPU側の描画処理を分けて考え、Shader Stage間でデータを受け渡す経験を得ました。

5.5 CSV駆動の簡易カットシーン制御

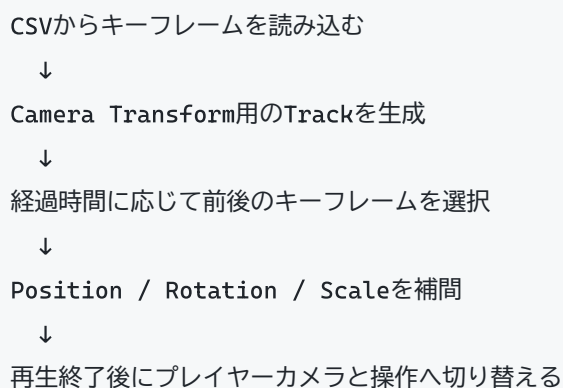
目的

ステージ開始時に、プレイヤー操作へ移る前の簡単なカメラ演出を再生する必要がありました。

判断と設計

短期間で最低限の演出を実現するため、汎用的な映像編集ツールではなく、Camera Transformの再生に対象を限定しました。時刻とPosition、Rotation、ScaleをCSVへ記録し、実行時に読み込んでキーフレーム間を補間するTrackTransformを実装しました。

処理フロー



結果

機能はCamera Transformの基本操作に限定されましたが、演出データをコードから分離し、時間軸に沿って再生する最小構成を実装しました。限られた期間内で、目的に必要な範囲へ仕様を絞る判断も経験しました。

5.6 レベル設計とゲームへの統合

ステージの進行、地形と障害物、敵の配置、戦闘の流れ、カメラ演出を含む1レベルを設計・実装しました。AIの基礎実装自体は担当外ですが、レベル内で敵を起動するタイミングや戦闘イベント、VFX、Sound、UIなどを組み合わせ、ゲームとして成立させるための統合を担当しました。

この作業を通して、フレームワークの設計だけでなく、そのAPIを利用して実際のコンテンツを制作し、不足している機能を基盤へフィードバックする流れを経験しました。

6. 成果

- 既製ゲームエンジンを使用せず、C++とDirectX 11による3Dゲームをチームで完成させました
- 個人として、アプリケーション起動、オブジェクト更新、描画、入力、3C、戦闘、演出までをつなぐ広い範囲を担当しました
- DirectX 11の描画パイプラインを一から構築し、プロジェクト内の全HLSL Shaderを設計・実装しました

- 自作フレームワーク上でGameplayを実装することで、基盤設計が実際のゲーム制作へ与える影響を確認しました
- 短期間の制約下で、粒子システムや簡易カットシーン制御など、必要な機能の範囲を定めて実装しました

7. 本プロジェクトで得た知識・能力

7.1 ゲームエンジン内部構造への理解

UnityやUnreal Engineで利用していたGameObject、Component、Transform階層、Controller、Camera、ライフサイクルなどについて、自ら最小構成を設計・実装しました。これにより、既製機能の使用方法だけでなく、それらが解決している問題と内部のデータフローを考えられるようになりました。

7.2 低レイヤーとGameplayを接続する能力

DirectX 11のリソースやPipeline Stateから、プレイヤー操作、射撃、VFX、UIまでを同じプロジェクト内で扱いました。この経験から、上位のゲーム仕様を実現するために下位基盤へ何が必要か、また基盤側の制約が操作感や演出へどう影響するかを意識するようになりました。

7.3 Application Programmerとして活かせる点

本プロジェクトでは、特定のエンジン機能だけを担当するのではなく、アプリケーションの起動、フレーム更新、オブジェクト管理、描画、入力、カメラ、Gameplay、演出の接続までを経験しました。そのため、ゲーム固有の要件を理解しながら、エンジンや共通基盤の制約を踏まえて実装する業務に活かせると考えています。

8. 技術的課題と今後の改善

本作では、限られた開発期間内でゲームを完成させることを優先したため、設計、機能、開発環境など、プロジェクト全体に多くの技術的課題が残りました。その中でも、プレイヤーの体験をより良くするという観点から、特に優先して改善したい内容を以下に挙げます。いずれも本作では未実装です。

実現したいこと	ゲーム体験への効果	実現方法
照準時のカメラ・遮蔽物制御	上方向への照準やADS時に、自機の車体や手前の地形・オブジェクトが目標を隠す状況を減らし、照準対象の視認性を保つ	照準方向に応じてCamera Offsetと距離を動的に調整する。さらに、Cameraから照準点までのRaycastで遮蔽物を検出し、一時的な半透明化やカメラ位置の補正を行う
FBXに対応したAsset Pipeline	利用できるモデル表現の幅を広げ、より情報量のあるキャラクターやステージを構築する	FBX SDKなどを利用し、階層構造、Material、Bone、Animationを含むデータを読み込むImporterを実装する
キャラクターAnimation System	状態や行動を動きで伝え、攻撃・被弾などに対する反応と臨場感を高める	Bone階層、Skinning、Animation Clipの再生・補間・遷移を実装する
Data-Drivenなゲーム構成	敵配置、難易度、カメラ、VFX、イベントの試行錯誤を容易にし、戦闘の手触りやレベル進行を継続的に調整する	EngineとGameの責務を整理し、レベル構成やGameplayパラメーターを外部データから設定できるようにする

これらを通して、基盤機能をゲーム上の見た目、操作へのフィードバック、遊びのテンポへ結び付け、プレイヤーが直接感じられる品質を高めたいと考えています。

9. 代表コード・資料

領域	代表ファイル
メインループ	Engine.cpp
オブジェクト／コンポーネント	GameObject.*、Component.*、Transform.*、SceneManager.*
DirectX 11基盤	Graphics.*、Rendering.*、RenderManager.*
描画抽象	Drawable.*、Mesh.*、Bindable.*、各Buffer／Shader／Stateクラス
Shader	プロジェクト内の全.hlsl／.hlslファイル
フォント／文字描画	FreeType.*、UIText.*
3C／Gameplay	PlayerController.*、PlayerTank.*、PlayerCamera.*
粒子システム	ParticleSystem.*、Particle_VS.hlsl、Particle_GS.hlsl、Particle_PS.hlsl
簡易カットシーン	Track.h、TrackTransform.*、Resource/Configuration/Sequence.csv
操作説明	Documents/3DTank_操作方法_JA.png